

WanderSync Travel Solutions

Plataforma de Empaquetamiento Turístico Dinámico

Documento Técnico de Arquitectura

Patrones Arquitectónicos Avanzados — Parcial 2 (Segundo Corte)

Equipo WanderSync

2026-10-10 · <https://github.com/Jorgealis/WanderSync-Travel-Solutions>

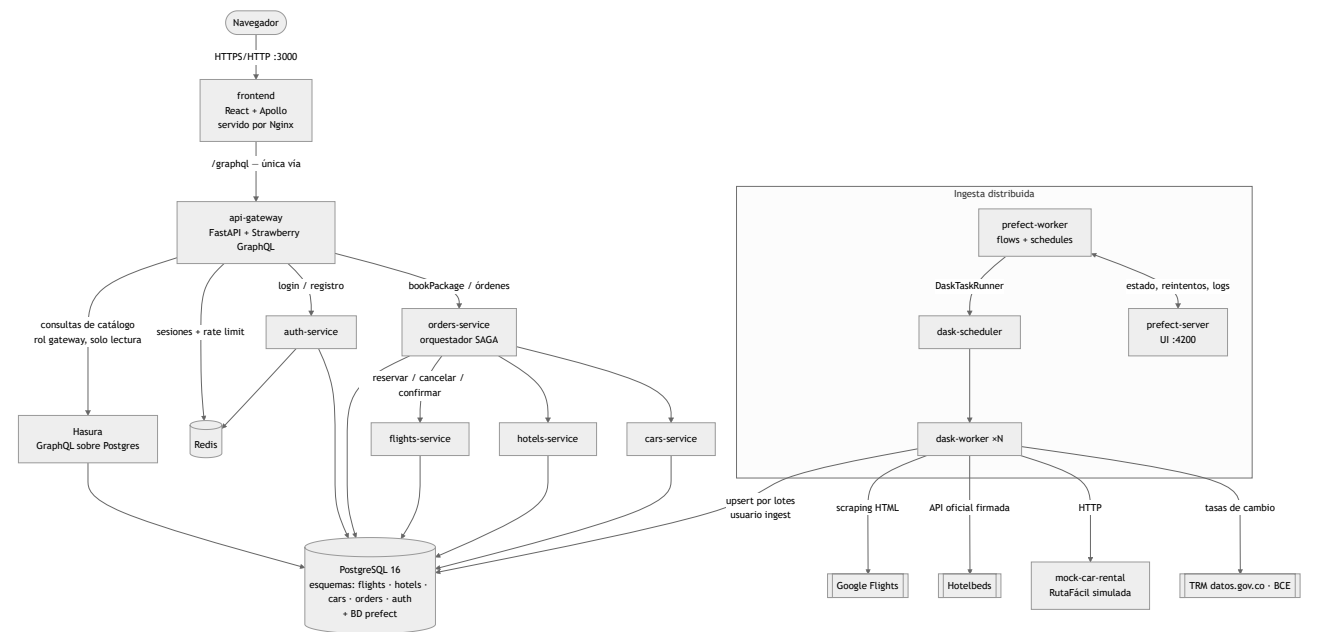
Contenido

1. Arquitectura
2. Patrón SAGA
3. Seguridad por diseño
4. Anexo A — Trazabilidad de requerimientos

1. Arquitectura

WanderSync vende paquetes turísticos dinámicos (vuelo + hotel + auto). Este documento describe los componentes, el despliegue, el flujo de datos y la justificación técnica de cada decisión. Documentos relacionados: [SAGA](#), [seguridad](#), [contratos](#) y [fuentes de datos](#).

1. Vista de componentes

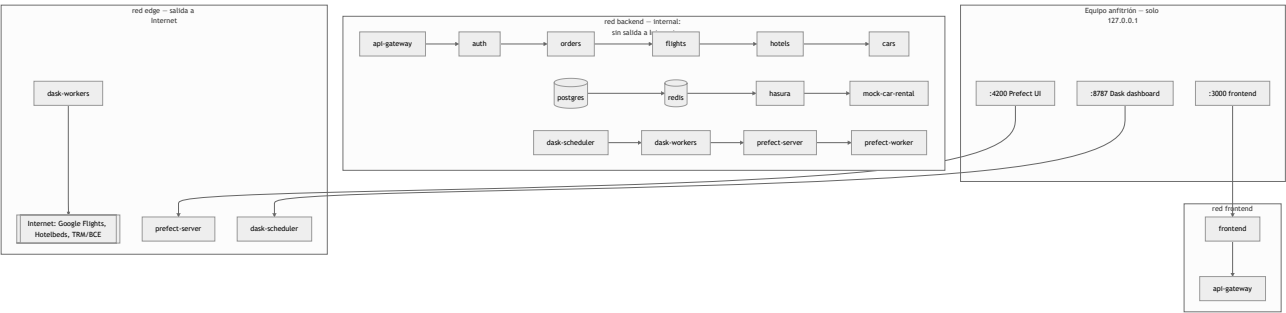


Servicio	Tecnología	Responsabilidad	Datos
frontend	React 19 + Vite + TypeScript + Apollo Client, servido por Nginx	Búsqueda, armado del paquete, checkout, línea de tiempo de la SAGA	—
api-gateway	FastAPI + Strawberry GraphQL	Único punto de entrada: consultas de catálogo, mutaciones de reserva, sesiones, rate limiting	Redis
auth-service	FastAPI + Argon2id	Registro, login con rotación de sesión, logout	esquema <code>auth</code> + Redis
flights-service, hotels-service, cars-service	FastAPI + SQLAlchemy async	Catálogo e inventario ; reservar/cancelar/confirmar idempotentes (participantes de la SAGA)	esquemas <code>flights</code> , <code>hotels</code> , <code>cars</code>
orders-service	FastAPI	Órdenes, pago simulado, facturación y orquestador SAGA	esquema <code>orders</code>
hasura	Hasura GraphQL Engine v2	GraphQL transparente sobre el catálogo (requisito 3.3)	lee los esquemas de catálogo
postgres	PostgreSQL 16	Persistencia; un esquema y un usuario por servicio	—
redis	Redis 7.4	Sesiones del lado del servidor y contadores de rate limiting	—
dask-scheduler + dask-worker xN	Dask Distributed	Ejecución paralela de scraping, normalización e ingesta	—
prefer-server + prefer-worker	Prefect 3	Flows, schedules, reintentos, observabilidad (UI)	base de datos <code>prefer</code> (mismo Postgres)

Servicio	Tecnología	Responsabilidad	Datos
mock-car-rental	FastAPI	Fuente simulada de autos (RutaFácil) con fallos, lentitud y paginación	—

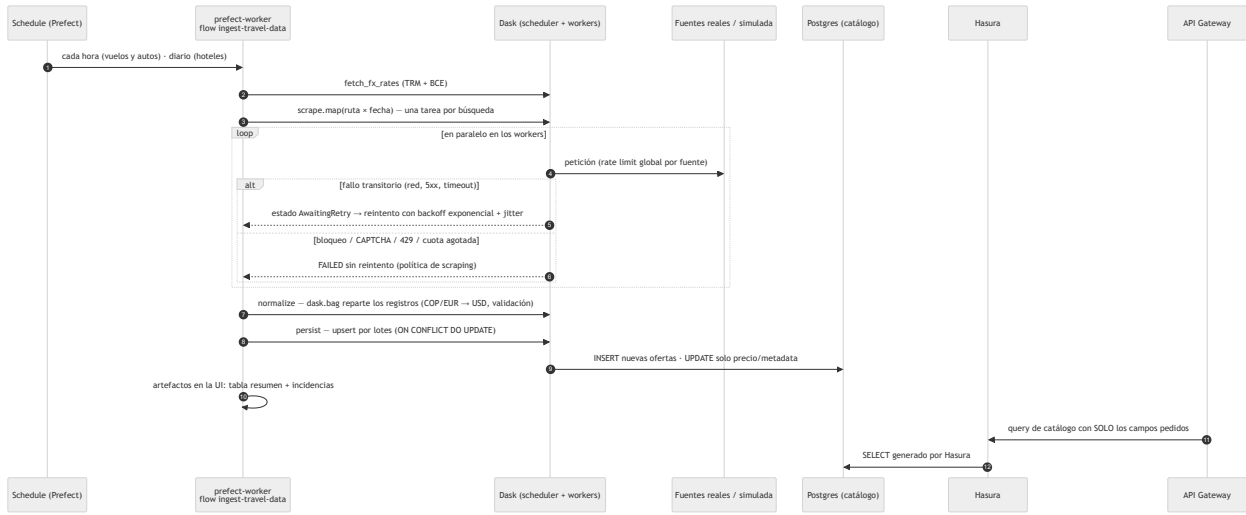
2. Vista de despliegue y redes

Todo se levanta con **un solo comando**: `docker compose up` (17 contenedores, todos con `healthcheck` y `depends_on: condition: service_healthy`). Las migraciones de cada servicio, la metadata de Hasura, los deployments de Prefect y la primera ingesta se aplican solos al arrancar.



- **backend es interna:** los microservicios, Postgres, Redis y Hasura no tienen salida a Internet ni puertos publicados.
- **edge** solo la usan los workers de Dask (scraping real) y los paneles de Prefect y Dask.
- `docker-compose.debug.yaml` publica Postgres, la consola de Hasura y el gateway directo para desarrollo o para la demo; no se usa por defecto.
- Endurecimiento: usuarios no-root, `cap_drop: ALL`, `no-new-privileges`, sistema de archivos de solo lectura en servicios sin estado (ver [seguridad \\$4](#)).
- Escalado: `docker compose up -d --scale dask-worker=4` (por defecto `DASK_WORKER_REPLICAS=3`).

3. Flujo de ingesta: fuentes → Prefect → Dask → Postgres → Hasura → Gateway



Etap	Qué hace	Dónde
Orquestación	Flow <code>ingest-travel-data</code> con subflows por catálogo; deployments con <code>schedule</code> registrados por <code>prefect.serve</code>	<code>data-pipeline/flows/ingest.py</code> , <code>flows/serve.py</code>
Distribución	<code>DaskTaskRunner</code> contra el scheduler externo: cada búsqueda es una tarea en un worker	<code>flows/ingest.py</code> (<code>dask_runner</code>)
Reintentos	<code>retries=3</code> , backoff exponencial con jitter, <code>retry_condition_fn</code> que solo reintenta fallos transitorios	<code>RETRY_POLICY</code> en <code>flows/ingest.py</code>

Etap	Qué hace	Dónde
Ritmo por fuente	<code>rate_limit("scraper-<fuente>")</code> : límite global entre todos los workers (p. ej. 1 petición/5 s a Google Flights)	<code>ensure_rate_limits</code>
Cuota	Hotelbeds: 45 de 50 peticiones diarias, contador compartido en el volumen <code>ingest-state</code>	<code>scrapers/quota.py</code>
Normalización	<code>dask.bag</code> reparte los registros en el clúster; conversión a USD con las tasas del día	<code>ingestion/normalize.py</code>
Persistencia	Upsert por lotes; el inventario solo se escribe al insertar (la SAGA lo gestiona después)	<code>ingestion/persist.py</code>
Observabilidad	Estados, reintentos, logs y artefactos (resumen e incidencias) en la UI de Prefect; tareas en el dashboard de Dask	UI :4200, dashboard :8787
Arranque en frío	Si un catálogo está vacío al arrancar, <code>serve.py</code> lanza la primera ingesta sola	<code>bootstrap_first_ingestion</code>

4. Fuentes de datos y política de scraping responsable

Catálogo	Fuente	Tipo	Ficha
Vuelos	Google Flights	Scraping de HTML real (aria-labels)	google-flights.md
Hoteles	Hotelbeds (entorno de evaluación)	API oficial con firma SHA-256	hotelbeds.md
Autos	RutaFácil	Servicio simulado (permitido por §3.1 del enunciado)	rutafacil-simulada.md

Política (se aplica en código, no solo en papel):

- Solo rutas que el `robots.txt` de la fuente **no prohíbe**.
- Nunca** se saltan CAPTCHAs ni protecciones anti-bot: un desafío o un 429 se trata como bloqueo y **no se reintenta**.
- Pocas peticiones y espaciadas: pausa mínima por fuente, límite global en Prefect aunque Dask tenga más hilos libres. (Google Flights exige un User-Agent de navegador, `SCRAPER_USER_AGENT`; con el de httpx responde "navegador no compatible".)
- Si una fuente falla o cambia su HTML, el flow termina de forma controlada, reporta las incidencias y el catálogo conserva los últimos datos válidos.

Registro de fuentes: se evaluaron más de 25 fuentes, una por una (`robots.txt` y una única petición de prueba). Kayak, Booking, Despegar, TripAdvisor, Rentalcars, Localiza y otras se descartaron por prohibición en `robots.txt` o por protecciones anti-bot. El registro completo, con fechas y motivos, está en [PROGRESO.md §1.4](#).

Limitaciones conocidas: el HTML de Google Flights puede cambiar sin aviso (las pruebas usan HTML real guardado en `data-pipeline/tests/fixtures/` para detectarlo); el entorno de evaluación de Hotelbeds limita a 50 peticiones diarias; las fuentes reales no publican cupos, así que el inventario inicial lo asigna WanderSync (configurable) y desde ahí lo gestiona la SAGA.

5. API Gateway GraphQL sin over-fetching

- Única vía:** Nginx sirve el frontend y reenvía `/graphql` al gateway (mismo origen); ningún otro servicio es accesible desde el navegador.
- Consultas complejas de consolidación:** `searchPackages` combina vuelo + hotel + auto de un mismo destino y fechas y calcula el precio estimado en una sola consulta.
- Sin over-fetching de extremo a extremo:** 1. el frontend pide solo lo que pinta (fragments `FlightCard`, `HotelCard`, `CarCard` y tipos generados con GraphQL Codegen desde el contrato `docs/contratos/schema.graphql`); 2. el gateway lee la selección de la consulta (`_selection_names`, incluyendo fragments) y le pide a Hasura **solo esas columnas** (lista blanca `FLIGHT_FIELDS`, `HOTEL_FIELDS`, `CAR_FIELDS`); 3. los `DataLoader` agrupan las búsquedas por ID (evitan N+1 en órdenes y paquetes).
- Mutaciones:** `register`, `login`, `logout`, `bookPackage`; `bookPackage` responde de inmediato con la orden `PENDING` y el frontend sigue la SAGA consultando `order(id)`.
- Protección:** límite de profundidad, rate limiting y CSRF (ver [seguridad](#)).

6. Justificación técnica del stack

Decisión	Alternativas	Por qué
Python 3.12 + FastAPI en todo el backend	Node/NestJS, Java/Spring	Dask y Prefect son nativos de Python: un solo lenguaje para servicios y pipeline, con Pydantic, SQLAlchemy y pytest compartidos
Microservicios con un esquema y un usuario de BD por servicio	Una BD por contenedor	Aislamiento real de datos (<i>database-per-service</i> lógico con GRANTs por columna) sin multiplicar contenedores
Strawberry GraphQL en el gateway	Apollo Server, Graphene	Tipado con dataclasses, DataLoader incluido, integración nativa con FastAPI
Hasura como capa GraphQL sobre Postgres	Supabase (pg_graphql/PostgREST)	Cumple "integración transparente con GraphQL" con metadata versionada en Git (<code>infra/hasura/metadata</code>), permisos por rol y columna, y corre como un solo contenedor. Supabase autoalojado exige ~10 contenedores (auth, storage, realtime...) que el proyecto no usa
SAGA orquestada	Coreografía con broker	Ver saga.md §6 : flujo explícito, estado persistido, recuperación y demostración más clara
Redis	Sesiones JWT sin estado	Sesiones del lado del servidor permiten regenerar y revocar el ID (Session Fixation) y contadores atómicos compartidos para el rate limiting
Dask Distributed	Celery, Spark	Requisito obligatorio; además <code>dask.bag</code> reparte la normalización y su dashboard muestra el reparto en vivo
Prefect 3 con serve	Airflow	Requisito obligatorio; reintentos con condición, rate limits globales, artefactos y UI sin necesidad de work pools
React + Apollo + Codegen	Vue, Angular	Consultas tipadas desde el contrato y caché normalizada; los fragments evitan pedir campos de más
Nginx sin privilegios	Servir desde Vite	Imagen mínima no-root, cabeceras de seguridad y proxy de <code>/graphql</code> en el mismo origen (cookies <code>SameSite=Strict</code>)
Fuente simulada para autos	Scraping con navegador automatizado	Las 10 fuentes de autos evaluadas prohíben el scraping o usan anti-bot; §3.1 del enunciado permite servicios simulados

7. Estructura del repositorio

—	docker-compose.yml	# despliegue completo (un comando)
—	docker-compose.debug.yml	# publica Postgres, consola de Hasura y gateway (desarrollo/demo)
—	services/	
—	api-gateway/	# GraphQL (Strawberry), sesiones, rate limiting
—	auth-service/	# Argon2id, rotación de sesión
—	flights-service/	hotels-service/ cars-service/ # catálogo + reservas (SAGA)
—	orders-service/	# órdenes, pago, factura, orquestador SAGA
—	mock-car-rental/	# fuente simulada de autos
—	data-pipeline/	# flows de Prefect, scrapers, normalización e ingesta (imagen de Dask)
—	frontend/	# React + Apollo + Nginx
—	libs/common/	# utilidades compartidas (reservas idempotentes, logging, migraciones)
—	infra/	# init de Postgres y metadata de Hasura
—	tests/	# e2e (SAGA), ingest (Prefect+Dask), security
—	scripts/	# .env, pruebas, locks, auditoría
—	docs/	# este documento, saga, seguridad, contratos, fuentes, guion de demo

2. Patrón SAGA

La reserva de un paquete (vuelo + hotel + auto + pago) toca cuatro servicios con bases de datos independientes, así que no existe una transacción ACID que la cubra. WanderSync la resuelve con una **SAGA orquestada**: un orquestador dentro de `orders-service` ejecuta los pasos en orden, guarda el estado de cada uno en Postgres y, si algo falla, ejecuta automáticamente las **transacciones de compensación** en orden inverso. Así desaparecen las "reservas huérfanas" del enunciado (§2.2).

Pieza	Archivo
Orquestador (pasos, reintentos, compensación)	<code>services/orders-service/app/saga.py</code> (<code>execute_saga</code> , <code>_run_step</code> , <code>_compensate</code>)
Recuperación tras un reinicio	<code>services/orders-service/app/saga.py</code> (<code>recover_pending_sagas</code>), invocada al arrancar en <code>app/main.py</code>
Inicio de la SAGA (orden + instancia en una transacción)	<code>services/orders-service/app/router.py</code> (<code>POST /orders</code>)
Reserva / cancelación / confirmación idempotentes en vuelos, hoteles y autos	<code>libs/common/wandersync_common/reservations.py</code> (router compartido por los tres servicios)
Estado persistido	tablas <code>orders.saga_instances</code> y <code>orders.saga_steps</code>
Mutación pública	<code>bookPackage</code> en <code>services/api-gateway/schema.py</code>
Línea de tiempo en vivo	<code>frontend/src/pages/OrderPage.tsx</code> (consulta <code>order { saga { steps } }</code> cada segundo)

1. Pasos y compensaciones

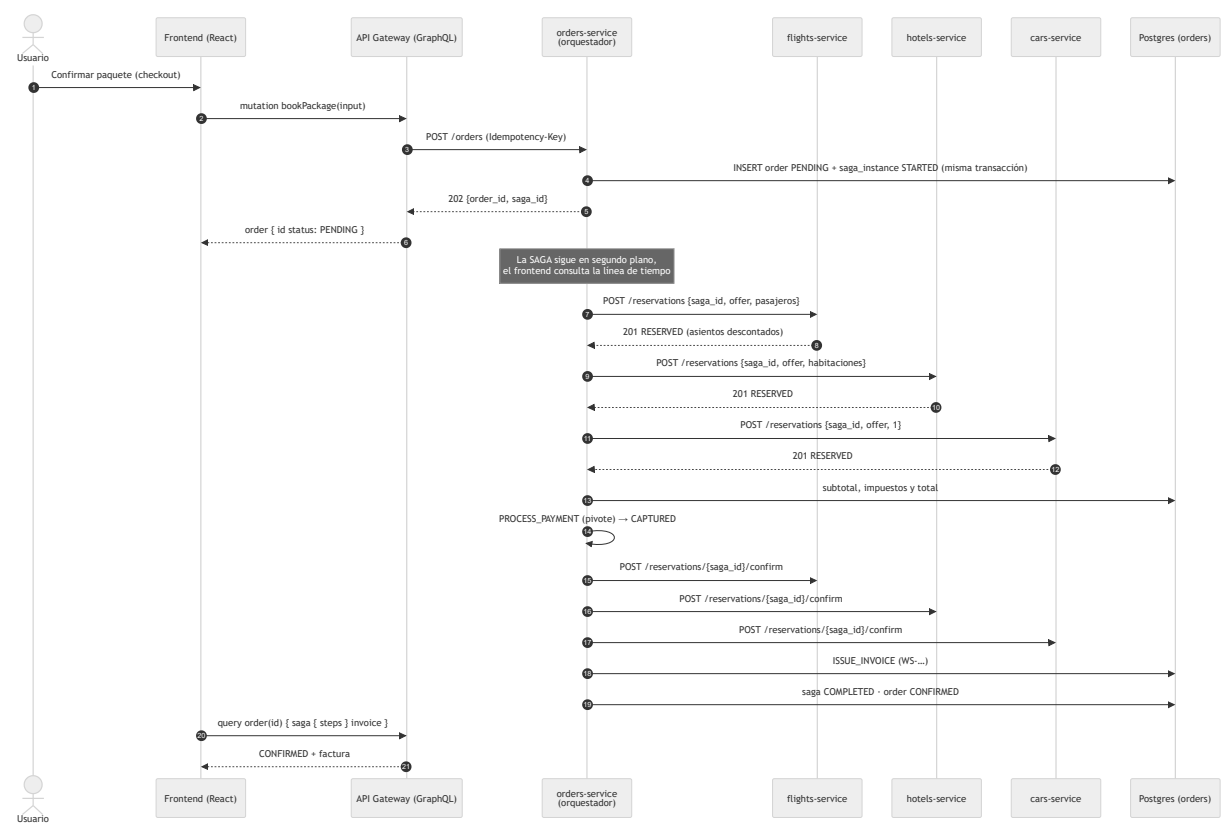
#	Paso	Tipo	Servicio	Acción	Compensación
1	<code>RESERVE_FLIGHT</code>	Compensable	flights-service	<code>POST /reservations</code> (descuenta asientos con <code>SELECT ... FOR UPDATE</code>)	<code>POST /reservations/{saga_id}/cancel</code> → devuelve los asientos
2	<code>RESERVE_HOTEL</code>	Compensable	hotels-service	<code>POST /reservations</code> (descuenta habitaciones)	<code>.../cancel</code> → devuelve las habitaciones
3	<code>RESERVE_CAR</code>	Compensable	cars-service	<code>POST /reservations</code> (descuenta 1 auto)	<code>.../cancel</code> → devuelve el auto
4	<code>PROCESS_PAYMENT</code>	Pivote	orders-service	Captura el pago (simulado)	Reembolso: <code>payments.status = REFUNDED</code>
5–7	<code>CONFIRM_FLIGHT/HOTEL/CAR</code>	Reintentable	los tres catálogos	<code>POST /reservations/{saga_id}/confirm</code>	— (no se compensan: se reintentan hasta completarse)
8	<code>ISSUE_INVOICE</code>	Reintentable	orders-service	Emite la factura <code>WS-...</code>	—

Reglas:

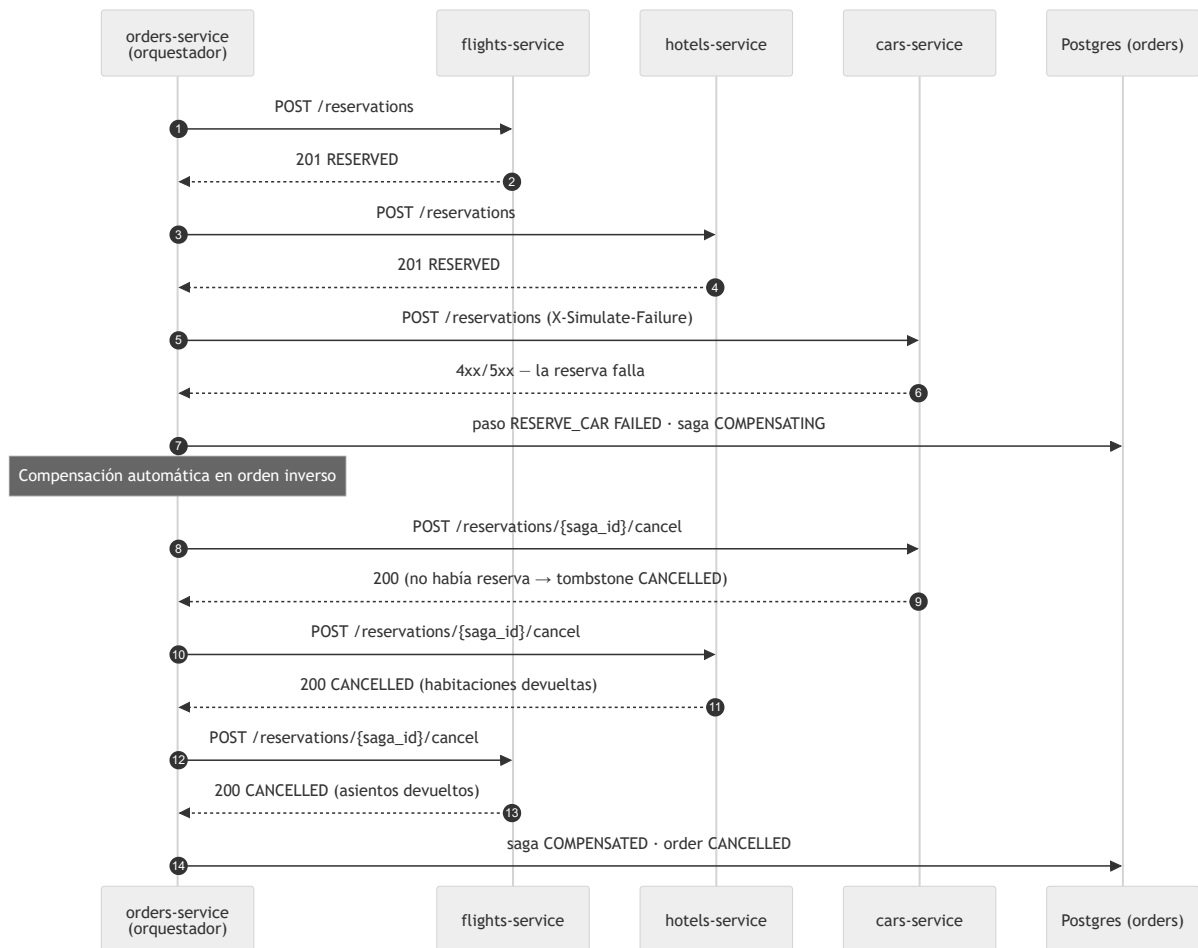
- **Antes del pivote**, si un paso falla se compensan **en orden inverso** todos los pasos de reserva, **incluido el que falló**: ante un timeout el orquestador no sabe si la reserva llegó a ejecutarse. Es seguro gracias al `tombstone` (abajo).
- **El pivote** (pago) es el punto de no retorno: si falla, se reembolsa (si llegó a capturarse) y se cancelan las tres reservas. Si tiene éxito, la SAGA ya no puede abortar.
- **Después del pivote**, las confirmaciones y la factura no se compensan: se reintentan sin límite (con backoff) hasta completarse.
- Errores: `4xx` = fallo de negocio → compensar sin reintentar; `5xx`, timeout o error de red = transitorio → reintentar con backoff exponencial hasta `SAGA_MAX_RETRIES` y luego compensar.

Estados: `saga_instances.status = STARTED → COMPLETED` O `STARTED → COMPENSATING → COMPENSATED`; la orden pasa de `PENDING` a `CONFIRMED` O `CANCELLED`.

2. Camino exitoso (happy path)

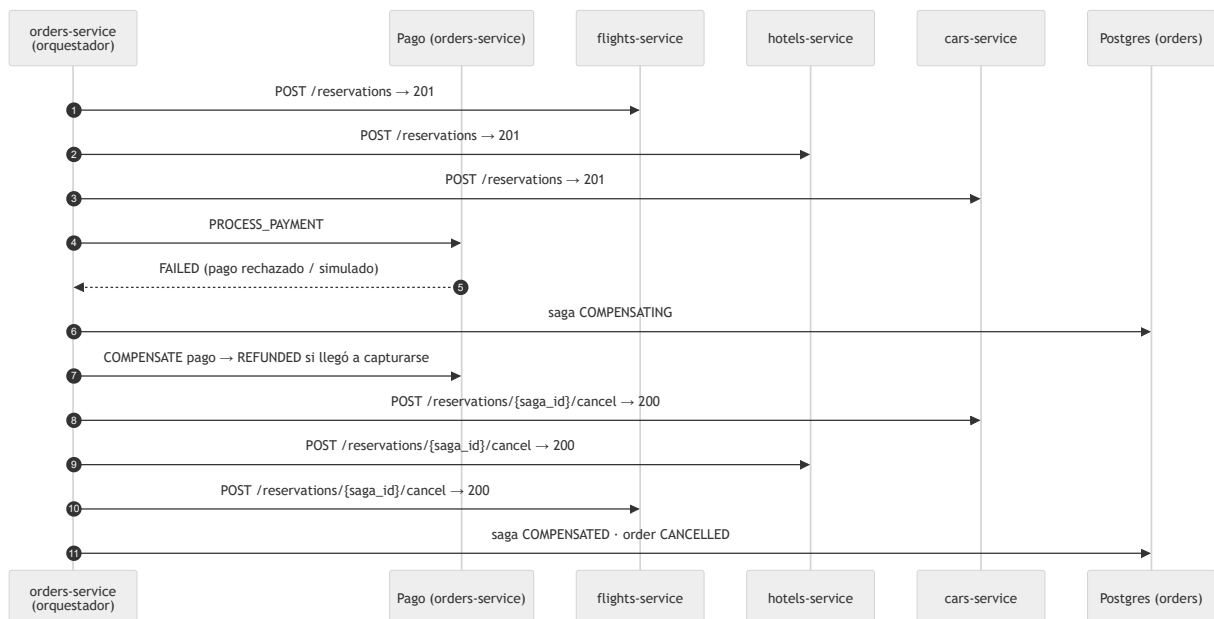


3. Fallo en autos: compensación de hotel y vuelo



El inventario de las tres ofertas vuelve exactamente al valor previo; lo comprueba `tests/e2e/test_saga.py`.

4. Fallo en el pago (pivote): reembolso y todas las cancelaciones



5. Garantías de consistencia

Problema	Solución	Dónde
Doble clic / reintento del cliente crea dos órdenes	Idempotency-Key en <code>POST /orders</code> : la misma clave devuelve la misma orden	<code>orders-service/app/router.py</code>
El orquestador reintenta una reserva que sí se hizo	Reserva única por <code>saga_id</code> en cada servicio: repetirla devuelve la existente	<code>libs/common/wandersync_common/reservations.py</code>
Una cancelación llega antes que una reserva retrasada	La cancelación deja un tombstone <code>CANCELLED</code> ; la reserva tardía responde <code>409 RESERVATION_CANCELLED</code> y no descuenta inventario	<code>reservations.py</code>
Dos SAGAs compiten por el último asiento	<code>SELECT ... FOR UPDATE</code> sobre la oferta; sin cupo → <code>409 INSUFFICIENT_AVAILABILITY</code> (fallo de negocio → compensar)	<code>reservations.py</code>
La ingesta pisa el inventario reservado	El upsert solo actualiza precio y metadata; el usuario de BD <code>ingest</code> no tiene permiso de <code>UPDATE</code> sobre las columnas de inventario	<code>data-pipeline/ingestion/persist.py</code> , migraciones
El orquestador se cae a mitad de la SAGA	Estado persistido paso a paso; al arrancar, <code>recover_pending_sagas</code> retoma las SAGAs <code>STARTED / COMPENSATING</code> y marca los pasos interrumpidos como <code>ORCHESTRATOR_RESTARTED</code>	<code>saga.py</code>
Una compensación falla por la red	Las compensaciones se reintentan sin límite (backoff con tope); son idempotentes	<code>_compensate</code> en <code>saga.py</code>

6. ¿Por qué orquestación y no coreografía?

Criterio	Orquestación (elegida)	Coreografía
Visibilidad del flujo	Un solo lugar (<code>saga.py</code>) define el orden; el estado completo está en <code>saga_steps</code> y se muestra en la línea de tiempo	El flujo está repartido en suscripciones a eventos de cada servicio
Compensación	El orquestador sabe qué pasos se ejecutaron y compensa en orden inverso	Cada servicio debe escuchar los eventos de fallo y deducir qué deshacer
Infraestructura	HTTP interno; no requiere broker de mensajes	Requiere un broker (Kafka/RabbitMQ) y garantías de entrega
Recuperación tras caídas	Estado persistido → se retoma al arrancar	Depende de la reentrega de eventos
Acoplamiento	Los servicios de catálogo no se conocen entre sí; solo exponen reservar/cancelar/confirmar	Cada servicio conoce los eventos de los demás
Riesgo	El orquestador es un punto central (mitigado: estado persistido y recuperación)	Ciclos de eventos difíciles de depurar

Con 4 participantes y un orden estricto (no se cobra antes de tener las tres reservas), la orquestación es más simple de razonar, de probar y de **demostrar**: la línea de tiempo del frontend muestra cada paso y cada compensación en vivo.

7. Cómo demostrarlo

- **Frontend:** en el checkout, el *Panel de demo · Simular fallo* en (`frontend/src/pages/CheckoutPage.tsx`) permite elegir `Ninguno`, `Vuelo`, `Hotel`, `Auto` o `Pago`. La página de la orden muestra la línea de tiempo con los pasos `EXECUTE` en verde, el fallo en rojo y las compensaciones `COMPENSATE`.
- **Prueba automatizada:** `tests/e2e/test_saga.py` ejecuta el happy path y los fallos en hotel, auto y pago a través del gateway, y verifica estados finales e inventario restaurado.
- **Base de datos:** `SELECT step, action, status, attempt FROM orders.saga_steps WHERE saga_id = '...' ORDER BY started_at;`
- **Configuración:** `ENABLE_FAULT_INJECTION=true` habilita la simulación (en producción va en `false` y el campo se ignora); `SAGA_STEP_DELAY_SECONDS=1` hace visible cada paso en la demo.

3. Seguridad por diseño

Este documento cubre los requisitos de ciberseguridad del enunciado (§5) y es la evidencia formal de las tareas 5.1–5.8 y 7.2. Todo lo que se afirma aquí se comprueba con una prueba automatizada o con un informe reproducible.

Requisito del enunciado	Control implementado	Evidencia
Session Fixation	El ID de sesión se genera en el servidor en cada login; el anterior se destruye	<code>tests/security/test_security.py::test_session_fixation_planted_id_is_replaced</code> , <code>test_each_login_rotates_and_revokes_previous_session</code>
Hashing robusto	Argon2id (<code>m=64 MiB</code> , <code>t=3</code> , <code>p=4</code>) con rehash automático si cambian los parámetros	<code>test_password_stored_with_argon2id</code> (lee el hash real de Postgres)
Rate limiting en autenticación	Login 5/min por IP, registro 3/min por IP	<code>test_login_rate_limited_per_ip</code> , <code>test_spoofed_forwarded_for_through_nginx_does_not_bypass_limit</code>
Rate limiting en checkout	<code>bookPackage</code> 3/min por usuario (gateway)	<code>test_checkout_rate_limited_per_user</code>
Rate limiting en pago	10/min por usuario en <code>orders-service</code> (defensa en profundidad)	<code>test_payment_rate_limited_in_orders_service</code>
Supply chain audit	<code>pip-audit</code> + <code>npm audit</code> + Trivy, informe antes/después, locks con hashes, CI y Dependabot	<code>docs/seguridad/antes/</code> , <code>docs/seguridad/despues/</code> , <code>.github/workflows/security.yml</code>

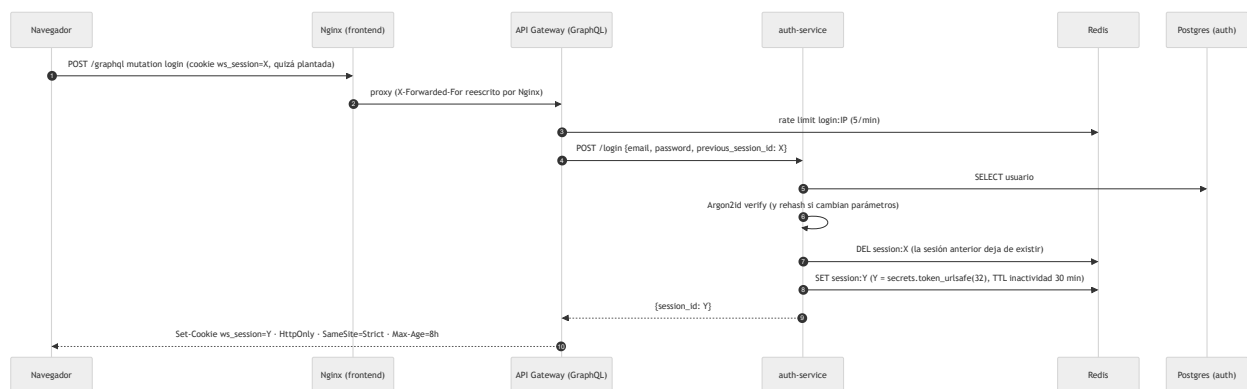
Ejecutar toda la suite contra el stack levantado (`docker compose up -d`):

```
python scripts/run_service_tests.py security
```

Resultado actual: **15 passed**.

1. Identidad y sesiones

1.1 Diseño



- **Session Fixation:** el ID nunca lo elige el cliente. En cada login se crea uno nuevo con `secrets.token_urlsafe(32)` (256 bits) y el que traía el navegador se destruye en Redis (`services/auth-service/app/router.py`, función `login`; el gateway envía `previous_session_id` en `services/api-gateway/schema.py`). Un ID plantado por un atacante queda inválido.
- **Revocación en el servidor:** `logout` borra la sesión en Redis (no basta con borrar la cookie).
- **Expiración:** 30 min de inactividad (TTL deslizante) y 8 h absolutas.
- **Cookie:** `HttpOnly` (no la lee JavaScript), `SameSite=Strict` y `Secure` configurable para HTTPS (`SESSION_COOKIE_SECURE=true` y nombre `__Host-ws_session` en producción).
- **CSRF:** además de `SameSite=Strict`, toda mutación exige la cabecera `X-WanderSync-CSRF: 1`, que un formulario de otro sitio no puede enviar.
- **Sin enumeración de usuarios:** mismo error (`INVALID_CREDENTIALS`) para correo inexistente y contraseña incorrecta; si el usuario no existe se verifica contra un hash señuelo para que el tiempo de respuesta sea el mismo.

1.2 Contraseñas: Argon2id

Parámetro	Valor	Variable
Algoritmo	Argon2id (ganador de la Password Hashing Competition, recomendado por OWASP)	—
Memoria	65 536 KiB (64 MiB)	<code>ARGON2_MEMORY_COST_KIB</code>
Iteraciones	3	<code>ARGON2_TIME_COST</code>
Paralelismo	4	<code>ARGON2_PARALLELISM</code>
Longitud de contraseña	10 a 128 caracteres	<code>PASSWORD_MIN_LENGTH</code> (<code>auth-service</code>)

El hash guardado tiene la forma `$argon2id$v=19$m=65536,t=3,p=4$<sal>$<hash>`: la prueba lo lee directamente de la tabla `auth.users`.

2. Protección de la superficie

2.1 Rate limiting

Contador atómico en Redis (script Lua `INCR + EXPIRE`), compartido por todas las réplicas del gateway. Al superar el límite el gateway responde el error GraphQL `RATE_LIMITED` con `retryAfter` y `orders-service` responde HTTP 429 con `Retry-After`.

Ruta	Límite	Clave	Dónde
<code>mutation login</code>	5 / minuto	IP	<code>services/api-gateway/schema.py</code> (<code>login</code>)
<code>mutation register</code>	3 / minuto	IP	<code>services/api-gateway/schema.py</code> (<code>register</code>)
<code>mutation bookPackage</code> (checkout)	3 / minuto	usuario	<code>services/api-gateway/schema.py</code> (<code>book_package</code>)
<code>POST /orders/{id}/payment</code>	10 / minuto	usuario	<code>services/orders-service/app/limiter.py</code>
Resto de operaciones	120 / minuto	IP	<code>get_context</code> del gateway

La IP sale de `X-Forwarded-For`, pero **Nginx la sobrescribe** con la IP real del cliente (`proxy_set_header X-Forwarded-For $remote_addr`), así que un atacante no puede rotar IPs falsas para saltarse el límite (prueba `test_spoofed_forwarded_for_through_nginx_does_not_bypass_limit`).

2.2 Otras defensas del gateway

- **Límite de profundidad** de consultas GraphQL (8 niveles) contra consultas abusivas.
- **Cabeceras** en Nginx: `Content-Security-Policy` estricta, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`.
- **Privacidad de órdenes**: un usuario no puede leer órdenes de otro (`test_orders_are_private_between_users`).
- **Hasura sin exposición**: sin puertos publicados, consola desactivada y rol `gateway` con permisos de solo lectura por columna.

3. Cadena de suministro

3.1 Herramientas y alcance

Herramienta	Versión	Qué audita
<code>pip-audit</code>	2.10.1	Inventario real de cada imagen Python (dependencias directas y transitivas instaladas)
<code>npm audit</code>	npm 10.9	Frontend: todas las dependencias y solo las de ejecución (<code>--omit=dev</code>)
Trivy	0.75.0	Cada imagen (propias y de terceros): paquetes del SO, paquetes de lenguaje y secretos incrustados

Reproducir (requiere las imágenes construidas):

```
docker compose build
python scripts/security_audit.py despues
```

Genera `docs/seguridad/<etiqueta>/auditoria.json` (evidencia completa) y `RESUMEN.md`.

3.2 Antes y después

Línea base (`docs/seguridad/antes/` , 2026-10-10 04:33 UTC) contra estado final (`docs/seguridad/despues/`):

Hallazgo	Antes	Después
<code>pip-audit</code> : vulnerabilidades Python (8 imágenes)	53 (pip 25.0.1 en todas; Strawberry 0.287.3 en el gateway)	0
<code>npm audit</code> : dependencias de ejecución (lo que llega al navegador)	0	0
<code>npm audit</code> : dependencias de desarrollo	10 HIGH (una cadena: <code>braces</code>)	10 HIGH — riesgo aceptado (§3.4)
Trivy imágenes propias: hallazgos con corrección disponible	14 por imagen (19 en el gateway)	0
Trivy frontend (Nginx)	1 HIGH (<code>tiff</code>)	0 en todas las severidades
Trivy: secretos en imágenes	0	0
Trivy Hasura	no se pudo analizar	analizado (ver §3.4)

3.3 Correcciones aplicadas (tarea 5.6)

1. **pip 25.0.1 → fuera de las imágenes**. En el build se usa pip 26.2.1. En la imagen final pip se desinstala: no se necesita en ejecución y arrastraba sus propias dependencias embebidas (`urllib3`, `msgpack`) con vulnerabilidades. La etapa `test` lo reinstala con `ensurepip`.
2. **Strawberry GraphQL 0.287.3 → 0.332.1** (5 advisories), con `graphql-core` 3.3.0.
3. **Locks con hashes SHA-256**: cada servicio tiene `requirements.lock` generado con `pip-compile --generate-hashes` (`python scripts/lock_requirements.py`) y las imágenes instalan con `pip install --require-hashes`. Si un paquete

publicado cambia (paquete comprometido), el build falla. El frontend usa `package-lock.json` con `npm ci`.

4. **Parches del sistema operativo:** `apt-get upgrade` (Debian) en las imágenes Python y `apk upgrade` (Alpine) en la de Nginx.
5. **Imágenes base fijadas** a versiones concretas (`python:3.12-slim`, `node:22.23.3-alpine`, `nginx-unprivileged:1.31.6-alpine`, `postgres:16.15-alpine`, `hasura v2.51.0`).

3.4 Riesgos aceptados

Hallazgo	Por qué se acepta	Mitigación
44 HIGH de Debian 13 en las imágenes Python (17 paquetes del SO, p. ej. <code>libsystemd0</code> , <code>login</code>)	Debian no ha publicado corrección ; <code>apt-get upgrade</code> ya instala todo lo disponible	Ninguno de esos binarios se ejecuta (solo corre <code>python / uvicorn</code>); contenedores sin capacidades, solo lectura y no-root. El CI vuelve a bloquear en cuanto exista la corrección (<code>ignore-unfixed</code>)
<code>braces</code> ≤ 3.0.3 (npm, 10 HIGH)	Solo en herramientas de desarrollo (GraphQL Codegen); no existe versión corregida y <code>npm audit fix</code> propone degradar Codegen	No llega al navegador (<code>npm audit --omit=dev = 0</code>). El CI bloquea vulnerabilidades moderadas en ejecución y críticas en desarrollo
Postgres: Go <code>stdlib</code> dentro de <code>gosu</code> (1 CRITICAL, 24 HIGH)	Binario de la imagen oficial; solo se usa al arrancar para cambiar de usuario	Red interna sin puertos publicados; proceso como uid 70 sin capacidades extra
Redis: <code>libssl3 / libcrypto3</code>	Corregido en Alpine, pendiente de nueva imagen oficial	Red interna; Redis no usa TLS en este despliegue; corre como uid 999 sin capacidades efectivas
Hasura: <code>openssl</code> , <code>curl</code> , <code>libxml2</code> (Ubuntu)	Imagen oficial pendiente de reconstrucción	Red interna, sin puertos ni consola; rol <code>gateway</code> de solo lectura

3.5 Automatización (tarea 5.8)

`.github/workflows/security.yml` se ejecuta en cada PR, en cada push a `prod / main` y cada lunes:

Job	Qué hace	Falla si...
<code>locks</code>	<code>scripts/check_locks.py</code>	<code>requirements.txt</code> cambió sin regenerar el lock (p. ej. un PR de Dependabot)
<code>pip-audit</code> (×8)	Audita cada <code>requirements.lock</code> con <code>--require-hashes</code>	Hay cualquier vulnerabilidad conocida
<code>npm-audit</code>	<code>npm ci</code> , audit y build (codegen + TypeScript estricto)	Vulnerabilidad ≥ moderada en ejecución o crítica en desarrollo
<code>pipeline-tests</code>	75 pruebas del pipeline (scrapers, normalización, cuota)	Alguna prueba falla
<code>trivy</code> (×6)	Construye la imagen y la escanea	CRITICAL/HIGH con corrección disponible o un secreto

`.github/dependabot.yml` abre PRs semanales agrupados para pip, npm, imágenes Docker y GitHub Actions.

4. Endurecimiento de contenedores (tarea 5.7)

Medida	Dónde	Cómo comprobarlo
Usuario no-root en todas las imágenes propias (<code>app</code> , Nginx uid 101)	Dockerfiles	<code>docker inspect -f '{{.Config.User}}' <contenedor></code>
<code>cap_drop: [ALL]</code> + <code>no-new-privileges</code> en los 17 contenedores (Postgres y Redis solo recuperan las capacidades que su entrypoint necesita para bajar de root)	<code>docker-compose.yml</code> (<code>x-hardened</code>)	<code>docker inspect -f '{{.HostConfig.CapDrop}}'</code>
Sistema de archivos de solo lectura (con <code>/tmp</code> en memoria) en los 8 servicios sin estado	<code>x-readonly</code>	<code>docker compose exec flights-service touch /app/x</code> → Read-only file system
Red <code>backend interna</code> (sin salida a Internet)	<code>networks.backend.internal: true</code>	desde <code>flights-service</code> no se resuelve ningún dominio externo

Medida	Dónde	Cómo comprobarlo
Red edge solo para lo que necesita Internet (workers de Dask: scraping) y los paneles	<code>docker-compose.yml</code>	desde <code>dask-worker</code> sí se conecta a Google Flights y Hotelbeds
Solo 3 puertos publicados y en <code>127.0.0.1</code> : frontend, Prefect UI, dashboard de Dask	<code>docker-compose.yml</code>	<code>docker compose ps</code>
Postgres, Hasura y el gateway directo solo con <code>docker-compose.debug.yml</code>	<code>docker-compose.debug.yml</code>	—
Sin pip ni herramientas de build en las imágenes finales (multi-stage)	Dockerfiles	<code>docker compose exec flights-service python -m pip → No module named pip</code>
Sin secretos en imágenes ni en Git (<code>.env</code> ignorado; Trivy secret scan = 0)	<code>.gitignore</code> , Trivy	<code>docs/seguridad/despues/RESUMEN.md</code>

Hallazgo corregido durante el endurecimiento: Redis se arrancaba con `sh -c 'exec redis-server ...'`, lo que se saltaba el paso del entrypoint oficial que cambia al usuario `redis`, así que **corría como root**. Ahora pasa por `docker-entrypoint.sh` y corre como uid 999 sin capacidades efectivas.

4. Anexo A — Trazabilidad de requerimientos

Este documento sigue el enunciado del Parcial 2 sección por sección. Para cada requisito indica:

- **Cómo se cumple:** la decisión de diseño.
- **Dónde está:** archivos y líneas para enseñar en la sustentación.
- **Cómo demostrarlo:** qué ejecutar o mostrar en vivo (los números de paso remiten a [demo-guion.md](#)).

Preparación común: stack levantado (`docker compose up -d`), navegador en `http://localhost:3000` , Prefect en `http://localhost:4200` (o el puerto de `PREFECT_HOST_PORT`) y Dask en `http://localhost:8787` .

§1. Tecnologías nucleares obligatorias

Tecnología	Cómo se cumple	Dónde está	Cómo demostrarlo
Docker Compose	17 contenedores, un <code>Dockerfile</code> por servicio, todo con <code>docker compose up</code>	<code>docker-compose.yml</code> , <code>services/*/Dockerfile</code> , <code>data-pipeline/Dockerfile</code> , <code>frontend/Dockerfile</code>	<code>docker compose ps</code> → 17 servicios <code>healthy</code> (paso 1)
GraphQL	API Gateway Strawberry (única vía del frontend) + Hasura sobre Postgres	<code>services/api-gateway/schema.py</code> , <code>docs/contratos/schema.graphql</code>	Pestaña <i>Network</i> del navegador: todo va a <code>/graphql</code> (paso 3)
Patrón SAGA	SAGA orquestada en <code>orders-service</code> con compensaciones automáticas	<code>saga.py:71</code> (<code>execute_saga</code>), <code>saga.py:359</code> (<code>_compensate</code>)	Checkout con fallo simulado en <i>Auto</i> → línea de tiempo (paso 5)
Dask	Scheduler + 3 workers ejecutan scraping, normalización (<code>dask.bag</code>) e ingesta	<code>ingest.py:61</code> (<code>DaskTaskRunner</code>), <code>ingest.py:175</code> (<code>normalize</code>)	Dashboard de Dask → <i>Status / Workers</i> durante una ingesta (paso 2)
Prefect	Flow <code>ingest-travel-data</code> con retries, schedules, rate limits y artefactos	<code>ingest.py:279</code> , <code>serve.py:46</code>	UI de Prefect → <i>Runs</i> y <i>Deployments</i> (paso 2)

§3. Web scraping e ingesta distribuida

3.1 Fuentes de datos

Cómo se cumple. Tres fuentes y tres tipos de ingesta:

Catálogo	Fuente	Tipo	Código
Vuelos	Google Flights	Scraping real de HTML	<code>scrapers/google_flights.py</code>
Hoteles	Hotelbeds	API oficial (entorno de evaluación, firma SHA-256)	<code>scrapers/hotelbeds.py</code>
Autos	RutaFácil	Servicio simulado (lo permite §3.1) con fallos 5xx, 429, lentitud y paginación	<code>services/mock-car-rental</code> , <code>scrapers/mock_car_rental.py</code>

Se evaluaron más de 25 fuentes con una política de scraping responsable: respeto de `robots.txt` , nunca saltar CAPTCHAs y ritmo limitado. Registro en [PROGRESO.md §1.4](#) y fichas en [docs/fuentes/](#).

Cómo demostrarlo. - En el frontend, la búsqueda muestra *Última sincronización* y precios reales del día. - En Prefect, la corrida de `vuelos-y-autos` muestra tareas `vuelos BOG-MDE ...` con sus logs. - Mostrar `docs/fuentes/google-flights.md` y explicar por qué Kayak y Booking se descartaron (`robots.txt` y anti-bot).

3.2 Procesamiento distribuido con Dask

Cómo se cumple. - La recolección corre **fuera** de los servicios de la aplicación: en los workers de Dask, lanzada por Prefect. Los microservicios nunca hacen scraping. - Cada búsqueda (ruta × fecha, ciudad × fecha × noches) es una **tarea independiente** en Dask (`scrape_flights.map(...)`). - La **limpieza y estructuración** se reparten con `dask.bag.from_sequence(...).map(normalizer)`: conversión de COP/EUR a USD, validación y valores por defecto.

Dónde está. - [ingest.py:144](#): tareas de scraping. - [ingest.py:175](#): normalización con `dask.bag`. - [ingestion/normalize.py](#). - `dask-scheduler` y `dask-worker` (con replicas) en [docker-compose.yml](#).

Cómo demostrarlo. - Dashboard de Dask durante una ingesta: barras de tareas y *Task Stream* con varios workers en paralelo. - `docker compose up -d --scale dask-worker=4` y `docker compose exec prefect-worker python -m flows.smoke` imprime el reparto, por ejemplo `{'4e86...': 4, '971b...': 2, 'b5fa...': 3, 'fcbc...': 3}`. - Prueba automatizada: `python scripts/run_service_tests.py ingest`.

3.3 Persistencia con integración GraphQL

Cómo se cumple. - Los workers escriben **directamente** en PostgreSQL 16 con un upsert por lotes, usando el usuario `ingest` con permisos mínimos. - **Hasura** expone ese catálogo como GraphQL de forma transparente: es el equivalente a Supabase con `pg_graphql` que menciona el enunciado. - El gateway consulta Hasura con el rol `gateway`, de solo lectura y con columnas permitidas.

Dónde está. - [ingestion/persist.py](#): upsert y regla del inventario. - [infra/hasura/metadata/databases/databases.yaml](#): tablas y permisos versionados. - `_selection_names` en [schema.py:288](#).

Cómo demostrarlo. - Con `docker compose -f docker-compose.yml -f docker-compose.debug.yml up -d`, abrir la consola de Hasura en <http://localhost:8080> y ejecutar una query sobre `flight_offers`. - Explicar que, por defecto, Hasura no publica puertos (seguridad).

§4. Requisitos arquitectónicos obligatorios

4.1 Dockerización y microservicios

Cómo se cumple. - Microservicios: **Vuelos, Hoteles, Autos, Órdenes/Facturación** (con pagos y el orquestador SAGA), **Gateway**, además de **Auth** y la fuente simulada. - Cada servicio tiene su propio Dockerfile *multi-stage*, su esquema de BD y su usuario de BD con privilegios solo sobre ese esquema. - Arranque con **un solo comando**: `healthchecks` y `depends_on: service_healthy`. Las migraciones (Alembic), la metadata de Hasura, los deployments de Prefect y la primera ingesta se aplican solos.

Dónde está. - [docker-compose.yml](#). - [services/](#). - `entrypoint.sh` de cada servicio: `alembic upgrade head`. - [infra/postgres/init](#): esquemas y roles. - [serve.py:84](#): primera ingesta automática.

Cómo demostrarlo. - `docker compose up -d` y luego `docker compose ps:todo healthy`. - Mostrar el árbol de [services/](#) y un Dockerfile: etapas `builder`, `runtime` y `test`, usuario no-root.

4.2 GraphQL como API Gateway unificado

Cómo se cumple. - **Exclusividad.** Nginx sirve el frontend y reenvía solo `/graphql` al gateway. Los microservicios están en la red interna `backend`, sin puertos publicados, así que el navegador no puede llegar a ellos. - **Consultas complejas de consolidación.** `searchPackages` combina vuelo + hotel + auto de un mismo destino y fechas y calcula el precio estimado. `searchFlights`, `searchHotels` y `searchCars` aceptan filtros: escalas, estrellas, categoría y presupuesto. - **Mutaciones de reserva.** `bookPackage` (y `register`, `login`, `logout`). - **Sin over-fetching**, de extremo a extremo: 1. el frontend pide solo lo que pinta, con fragments y tipos generados (GraphQL Codegen); 2. el gateway traduce la selección del cliente a una query de Hasura **con solo esas columnas**; 3. los `DataLoader` evitan el problema N+1.

Dónde está. - [schema.py:606](#): `search_packages`. - [schema.py:288](#): `_selection_names`, que traduce la selección a columnas. - `FLIGHT_FIELDS` en [schema.py:215](#). - [frontend/src/graphql/operations.ts](#): fragments `FlightCard`, `HotelCard` y `CarCard`. - [frontend/nginx.conf](#): proxy de `/graphql`.

Cómo demostrarlo (paso 3 del guion). - En *DevTools* → *Network*, filtrar `graphql` y abrir la petición `SearchOffers`: el `payload` trae solo los campos de los fragments y la respuesta trae exactamente esos campos. - Comparar con el tipo `FlightOffer` del contrato, que tiene más campos de los que se piden.

4.3 Patrón SAGA

Cómo se cumple. - **SAGA orquestada:** `RESERVE_FLIGHT` → `RESERVE_HOTEL` → `RESERVE_CAR` → `PROCESS_PAYMENT` (pivote) → confirmaciones → factura. - Ante un fallo antes del pivote, la compensación es **automática y en orden inverso**. Si el auto falla, se cancelan auto, hotel y vuelo, y el inventario vuelve a su valor previo. - Consistencia: - idempotencia por `saga_id` y por `Idempotency-Key`; - *tombstones* para cancelaciones que llegan antes que la reserva; - `SELECT ... FOR UPDATE` contra la sobreventa; - recuperación tras un reinicio del orquestador.

Dónde está. - `saga.py:71`: `execute_saga`, los pasos. - `saga.py:177`: `_run_step`, reintentos y registro de cada intento. - `saga.py:359`: `_compensate`. - `libs/common/wandersync_common/reservations.py`: reserva, cancelación y confirmación idempotentes. - Diagramas en `saga.md`.

Cómo demostrarlo (pasos 4 y 5 del guion). - **Happy path:** checkout con *Ninguno*. La línea de tiempo muestra los 8 pasos en verde, la orden queda **Confirmada** y aparece la factura `WS-...`. - **Compensación:** checkout con *Auto*. `RESERVE_CAR` falla en rojo y aparecen *Compensación: Reserva del auto, ...del hotel y ...del vuelo*. La orden queda **Cancelada (compensada)**. - **Inventario restaurado:** los asientos, habitaciones y autos disponibles de la oferta vuelven al valor anterior. Se ve buscando de nuevo la misma oferta, y lo verifica `tests/e2e/test_saga.py`. - Prueba automatizada: `tests/e2e/test_saga.py` (happy path + fallos en hotel, auto y pago).

4.4 Computación distribuida con Dask

Ver §3.2. Además: - el escalado horizontal se hace con `--scale dask-worker=N`; - los workers comparten un límite de ritmo **global** por fuente (Prefect `rate_limit`), así que más workers no significa más presión sobre los sitios reales.

4.5 Orquestación y observabilidad con Prefect

Cómo se cumple. - **Flow** `ingest-travel-data`, con subflows por catálogo y un `DaskTaskRunner` que envía las tareas al clúster Dask. - **Retries explícitos:** - `retries=3`, backoff exponencial y jitter; - `retry_condition_fn` que **solo** reintenta fallos transitorios (red, 5xx, timeout); - los bloqueos (CAPTCHA, 429) y la cuota agotada no se reintentan, por la política de scraping. - **Fallos simulados** para la demo: parámetro `simulate_fault_rate`. - **Schedules:** vuelos y autos cada hora, hoteles a diario. **Artefactos:** tabla resumen e incidencias de cada ingest.

Dónde está. - `ingest.py:65`: `retry_only_transient`. - `ingest.py:76`: `RETRY_POLICY`. - `ingest.py:279`: el flow. - `serve.py`: deployments y schedules.

Cómo demostrarlo (paso 2 del guion). - En la UI de Prefect: *Deployments* → *vuelos-y-autos* → *Run* → *Custom run* con `catalogs=["cars"]` y `simulate_fault_rate=0.4`. - En la corrida se ven tareas en **AwaitingRetry / Retrying** y luego **Completed**, y la pestaña *Artifacts* muestra el resumen. - Prueba: `python scripts/run_service_tests.py ingest`. Por ejemplo, la última ejecución dio *23 de 30 búsquedas necesitaron reintentos (máx. 4 intentos)*.

§5. Ciberseguridad por diseño

Documento completo: `seguridad/README.md`. Prueba: `python scripts/run_service_tests.py security` → **15 passed**.

Session Fixation + hashing robusto

Cómo se cumple. - El ID de sesión **siempre** lo genera el servidor (`secrets.token_urlsafe(32)`) en cada login, y el anterior se **destruye** en Redis. - Cookie `HttpOnly` y `SameSite=Strict`; además, cabecera CSRF obligatoria en las mutaciones. - Contraseñas con **Argon2id** (`m=64 MiB`, `t=3`, `p=4`).

Dónde está. - `auth-service/app/router.py:95`: `create_session`, que destruye `previous_session_id` y crea un ID nuevo. - `auth-service/app/router.py:31`: `PasswordHasher` `Argon2id`. - `schema.py:891`: `login` en el gateway.

Cómo demostrarlo. - DevTools → Application → Cookies: anotar `ws_session` antes de iniciar sesión, iniciar sesión y ver que el valor **cambió**. - Mostrar el hash en la BD con el compose de depuración: `SELECT password_hash FROM auth.users LIMIT 1;` → `$argon2id$v=19$m=65536,t=3,p=4$...` - Pruebas `test_session_fixation_planted_id_is_replaced` y `test_password_stored_with_argon2id`.

Rate limiting en rutas sensibles

Cómo se cumple.

Ruta	Límite	Clave
Login	5/min	IP
Registro	3/min	IP
Checkout (bookPackage)	3/min	usuario
Pago (orders-service)	10/min	usuario

Los contadores son atómicos en Redis. Nginx reescribe `X-Forwarded-For`, así que el límite no se puede evadir falsificando la IP.

Dónde está. - [services/api-gateway/limiter.py](#). - [services/orders-service/app/limiter.py:37](#).

Cómo demostrarlo. - Intentar iniciar sesión con una contraseña incorrecta 6 veces seguidas: la sexta muestra *"Demasiados intentos. Espera un momento y vuelve a intentarlo."* (`RATE_LIMITED`). Hacerlo al final de la demo, porque bloquea el login desde esa IP durante un minuto. - Pruebas `test_login_rate_limited_per_ip`, `test_checkout_rate_limited_per_user` y `test_payment_rate_limited_in_orders_service`.

Supply chain security

Cómo se cumple. - Auditoría formal con pip-audit, npm audit y Trivy sobre lo que realmente se despliega, con informe **antes y después**: - Python: 53 vulnerabilidades → **0**; - hallazgos corregibles en las imágenes propias → **0**; - frontend → 0. - *Lockfiles* con hashes SHA-256 instalados con `--require-hashes`, que evitan paquetes alterados. - CI en GitHub Actions y Dependabot.

Dónde está. - [docs/seguridad/antes/RESUMEN.md](#) y [docs/seguridad/despues/RESUMEN.md](#), con su JSON de evidencia. - [scripts/security_audit.py](#). - `services/*/requirements.lock`. - [.github/workflows/security.yml](#).

Cómo demostrarlo. - Abrir los dos `RESUMEN.md` lado a lado. - Mostrar la pestaña *Actions* de GitHub con el workflow `security` en verde: 17 jobs. - Explicar los riesgos aceptados (§3.4 del documento de seguridad).

§6. Entregables

Entregable	Dónde
Repositorio limpio con microservicios, frontend, Dockerfile por servicio y <code>docker-compose.yml</code>	Rama <code>prod</code> de https://github.com/Jorgealis/WanderSync-Travel-Solutions
Documento técnico: diagramas de arquitectura, secuencias SAGA (éxito y compensación) y justificación técnica	documento-tecnico.pdf (fuentes: arquitectura.md , saga.md , seguridad/README.md)
Demostración en vivo (a) Prefect, (b) Dask, (c) GraphQL desde el frontend, (d) fallo SAGA con compensación	demo-guion.md , pasos 2, 2, 3 y 5

§7. Rúbrica: evidencia por criterio

Criterio (peso)	Qué pide el nivel <i>Excelente</i>	Evidencia en WanderSync
Arquitectura y Dockerización (15 %)	Despliegue automatizado con <code>docker compose up</code> ; microservicios aislados y bien configurados	17 contenedores <i>healthy</i> sin pasos manuales; un esquema y un usuario de BD por servicio; red interna; migraciones, Hasura, Prefect y primera ingesta automáticos
API Gateway GraphQL y persistencia (15 %)	Gateway funcional, consultas y mutaciones sin over-fetching, integración fluida con la BD	Selección del cliente → columnas en Hasura; fragments y Codegen; DataLoader; <code>searchPackages</code> ; Hasura sobre Postgres
SAGA y consistencia (25 %)	Happy path impecable y compensaciones automáticas ante fallos simulados	4 puntos de fallo seleccionables desde la UI; línea de tiempo en vivo; idempotencia, tombstones y recuperación; <code>test_saga.py</code> en verde
Dask + Prefect (25 %)	Workers de Dask con scraping/ingesta asíncrona integrados con Prefect, retries y monitoreo	Scraping real en Dask lanzado por flows de Prefect; retries condicionados y fallos simulables; artefactos; dashboard de Dask; escalado; <code>test_ingest_e2e.py</code> en verde
Ciberseguridad y resiliencia (20 %)	Session Fixation, Argon2id/bcrypt, rate limiting y reporte formal de supply chain	15 pruebas de seguridad; Argon2id; 4 límites; auditoría antes/después; CI + Dependabot; contenedores endurecidos

Resumen de pruebas automatizadas

Suite	Comando	Resultado
Pipeline (scrapers, normalización, cuota, persistencia)	<code>python scripts/run_service_tests.py pipeline</code>	75 passed
Contrato de reservas (vuelos, hoteles, autos)	<code>python scripts/run_service_tests.py flights hotels cars</code>	3 × 14 passed
Seguridad	<code>python scripts/run_service_tests.py security</code>	15 passed
E2E ingesta (Prefect + Dask)	<code>python scripts/run_service_tests.py ingest</code>	5 passed
E2E SAGA (gateway → SAGA → inventario)	<code>python -m pytest tests/e2e/test_saga.py</code>	1 passed (4 escenarios)
CI (GitHub Actions)	<code>push a prod</code>	17 jobs en verde